

עבודה מסכמת בקורס סמינר בהנדסת תוכנה 157119.5785

מגישות: שרה בורגן גרינפלד, מיכל מילר

א. מבוא – תקציר ורקע

בשנים האחרונות חלה עלייה משמעותית בשימוש בשירותי ענן ובמערכות מקוונות, אשר הפכו לחלק בלתי נפרד מחיי היום-יום של ארגונים ומשתמשים פרטיים. לצד היתרונות הרבים של מחשוב ענן, כגון זמינות גבוהה, גישה מרחוק וחסכון בעלויות, מתעוררים גם אתגרי אבטחת מידע משמעותיים. אחד האיומים המרכזיים והנפוצים ביותר הוא מתקפות מניעת שירות מבוזרות (Distributed Denial of Service (DDoS).

מתקפת DDoS מתבצעת כאשר תוקף משתלט על מספר רב של מחשבים או התקנים ברשת, המכונים לעיתים "זומבים" או Botnet, ומשתמש בהם כדי לשלוח כמות עצומה של בקשות לשרת מסוים בו־זמנית. עומס זה גורם לכך שהשרת אינו מסוגל לספק שירות למשתמשים לגיטימיים, ולעיתים אף מוביל לקריסת המערכת כולה. ניתן לדמות מצב זה לכביש מהיר שאליו נכנסים בבת אחת מיליוני כלי רכב, עד שהתנועה נעצרת לחלוטין.

המאמר שבו עוסקת עבודה זו מציג מחקר בתחום זיהוי מתקפות DoS/DDoS באמצעות אלגוריתמים של למידת מכונה Machine Learning. הבעיה המרכזית שהחוקרים מנסים לפתור היא כיצד להבחין בצורה מהירה ומדויקת בין תעבורת רשת רגילה לבין תעבורה זדונית, בזמן אמת ובאחוזי שגיאות נמוך ככל האפשר. בעיה זו מעניינת וחשובה במיוחד משום שמתקפות DDoS הפכו בשנים האחרונות לנפוצות ומתוחכמות יותר, והן גורמות לנזקים כלכליים ותפעוליים משמעותיים לארגונים ברחבי העולם.

במסגרת המחקר נבחנו מספר אלגוריתמים של למידת מכונה לצורך זיהוי מתקפות, ביניהם Naive Bayes, Logistic Regression, Random Forest ו־XGBoost. החוקרים השתמשו במאגר הנתונים CICIDS2017, הנחשב למאגר מוכר ואמין בתחום אבטחת המידע, וביצעו תהליך של ניקוי נתונים, בחירת מאפיינים ואימון מודלים לצורך השוואת ביצועים.

מטרת העבודה שלנו היא לשחזר את מתודולוגיית המחקר ואת תוצאותיו בפועל, באמצעות מימוש עצמאי של המערכת המתוארת במאמר. במסגרת העבודה יושם דגש על עיבוד הנתונים, בחירת המאפיינים הרלוונטיים, אימון המודלים והערכת הביצועים שלהם, תוך השוואה לתוצאות שהוצגו במאמר המקורי.

ב. שיטות העבודה

במאמר נבחרה גישה המבוססת על Machine Learning לצורך זיהוי מתקפות DoS/DDoS בתעבורת רשת. מטרת השיטה היא לאמן מודלים חכמים אשר ילמדו לזהות דפוסים המאפיינים תעבורה זדונית, ולהבחין בינה לבין תעבורה תקינה בזמן אמת.

החוקרים בחרו להשתמש במאגר הנתונים CICIDS2017, הכולל תעבורת רשת אמיתית לצד סוגים שונים של מתקפות סייבר. מתוך המאגר נבחרו קבצי הנתונים של יום רביעי ושל יום שישי אחר הצהריים, אשר מכילים מתקפות DoS ו-DDoS.

תהליך העבודה חולק למספר שלבים מרכזיים. תחילה בוצע עיבוד מקדים לנתונים, שכלל ניקוי ערכים לא חוקיים כגון NaN ו-Infinity, המרת התוויות לבעיה בינארית Benign או Attack, וכן נרמול הנתונים באמצעות שיטת Z-score. מטרת הנרמול היא להביא את כל המאפיינים לאותה סקאלה, כך שהמודלים יוכלו ללמוד בצורה יעילה ומדויקת יותר.

לאחר מכן בוצעה בחירת מאפיינים. במקום להשתמש בכל 78 המאפיינים הקיימים במאגר, נבחרו שמונה מאפיינים מרכזיים שנמצאו כבעלי ההשפעה הגדולה ביותר על יכולת הזיהוי של המערכת. בין המאפיינים שנבחרו ניתן למצוא:

- Fwd Packet Length Max
- Flow IAT Mean
- Flow IAT Std
- Bwd Packet Length Mean

המאמר בחן ארבעה אלגוריתמים שונים של למידת מכונה:

- Naive Bayes (NB)
- Logistic Regression (LR)
- Random Forest (RF)
- XGBoost

מטרת ההשוואה הייתה לבדוק איזה מודל מצליח לזהות מתקפות בצורה המדויקת ביותר ובשיעור אזעקות שווא נמוך ככל האפשר.

אלגוריתם Naive Bayes הוא מודל הסתברותי פשוט ומהיר יחסית, המתבסס על משפט בייס. אלגוריתם Logistic Regression הוא מודל לינארי המשמש לבעיות סיווג בינאריות. אלגוריתם Random Forest מבוסס על אוסף של עצי החלטה, ומשלב ביניהם לצורך קבלת החלטה יציבה ומדויקת יותר. לעומתם XGBoost, הוא אלגוריתם מתקדם המבוסס על עצי החלטה ולמידת חיזוק, והוא נחשב לאחד האלגוריתמים החזקים והנפוצים כיום בתחום למידת המכונה.

במסגרת העבודה שלנו יושמו האלגוריתמים באמצעות שפת Python ובספריות:

- Scikit-Learn
- XGBoost
- Pandas
- NumPy

הקוד נכתב באופן עצמאי על בסיס המתודולוגיה המתוארת במאמר, מאחר והמאמר המקורי לא סיפק קוד מוכן למימוש. במהלך העבודה בוצעו התאמות

שונות לצורך שחזור התוצאות, כולל טיפול בשמות עמודות, ניקוי הנתונים, כוונון פרמטרים בסיסי ובחירת מאפיינים בהתאם למאמר.

בנוסף לשיטות שנבחרו במאמר, קיימות שיטות חלופיות נוספות לזיהוי מתקפות סייבר, כגון:

- Support Vector Machine (SVM)
- Deep Neural Networks (DNN)
- Convolutional Neural Networks (CNN)
- Recurrent Neural Networks (RNN)

שיטות אלו עשויות להציג ביצועים גבוהים, אך לרוב דורשות זמן אימון ארוך יותר ומשאבי חישוב גדולים יותר. מסיבה זו, המחקר התמקד באלגוריתמים קלאסיים ויעילים יותר, ובעיקר ב־ XGBoost אשר סיפק את התוצאות הטובות ביותר במחקר.

ג. הנתונים

במחקר נעשה שימוש במאגר הנתונים CICIDS2017, אשר פותח על ידי Canadian Institute for Cybersecurity ונחשב לאחד ממאגרי הנתונים המוכרים והנפוצים ביותר בתחום זיהוי מתקפות סייבר ומערכות IDS. המאגר נבנה במטרה לדמות תעבורת רשת אמיתית בסביבה מודרנית, והוא כולל גם תעבורה תקינה וגם סוגים שונים של מתקפות סייבר. המאגר זמין באינטרנט בקישור הבא:

<https://www.unb.ca/cic/datasets/ids-2017.html>

במסגרת העבודה נעשה שימוש בשני קבצים מתוך המאגר:

- Wednesday-workingHours.pcap_ISCX.csv
- Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv

קבצים אלו נבחרו משום שהם כוללים מתקפות DoS ו־DDoS, אשר מהוות את מוקד המחקר.

לאחר איחוד שני הקבצים התקבל מאגר נתונים בגודל של 918,448 רשומות ו־79 עמודות. כל רשומה מייצגת זרימת תעבורת רשת, כאשר כל עמודה מתארת מאפיין מסוים של אותה תעבורה, כגון:

- אורך חבילות מידע
- זמני מעבר בין חבילות
- מספר חבילות שנשלחו
- קצב תעבורה
- דגלי פרוטוקול שונים

הנתונים במאגר חולקו לשתי מחלקות עיקריות:

- תעבורה תקינה (Benign)

• תעבורה זדונית מסוג DoS/DDoS

במאגר המקורי הופיעו מספר סוגי מתקפות שונים, כגון:

- DoS Hulk
- DDoS
- DoS GoldenEye
- DoS Slowloris
- DoS Slowhttptest
- Heartbleed

בשלב עיבוד הנתונים אוחדו כל סוגי ההתקפות למחלקה בינארית אחת בשם "Attack", כך שהבעיה הפכה לבעיית סיווג בינארית:

- 0 תעבורה תקינה
- 1 מתקפה

לאחר ההמרה התקבלה החלוקה הבאה:

- 537,749 רשומות של תעבורה תקינה
- 380,699 רשומות של מתקפות

כחלק מהכנת הנתונים בוצעו מספר פעולות ניקוי ועיבוד מקדים:

1. ניקוי שמות עמודות והסרת רווחים מיותרים.
2. הסרת ערכים לא חוקיים כגון NaN ו-Infinity.
3. הסרת מאפיינים שאינם תורמים לתהליך הלמידה, בהתאם למאמר.
4. נרמול הנתונים באמצעות שיטת Z-score, כדי להביא את כל המאפיינים לסקאלה אחידה.

בנוסף, במקום להשתמש בכל 78 המאפיינים שבמאגר, נבחרה תת-קבוצה של 8 מאפיינים מרכזיים אשר נמצאו כבעלי ההשפעה הגבוהה ביותר על תהליך הזיהוי. בחירה זו בוצעה בהתאם למתודולוגיה המתוארת במאמר.

לצורך אימון והערכת המודלים חולקו הנתונים לשתי קבוצות:

- 70% נתוני אימון (Training Set)
- 30% נתוני בדיקה (Testing Set)

חלוקה זו מאפשרת לבדוק האם המודל מצליח ללמוד דפוסים כלליים מתוך הנתונים, ולא רק "לזכור" את דוגמאות האימון. בנוסף, נעשה שימוש ב-stratified split, כדי לשמור על יחס דומה בין מחלקות הנתונים גם בקבוצת האימון וגם בקבוצת הבדיקה.

ד. תוצאות

לאחר השלמת שלבי עיבוד הנתונים, בחירת המאפיינים ואימון המודלים, בוצעה הערכה של ארבעת האלגוריתמים שנבחרו במאמר:

- Naive Bayes
- Logistic Regression
- Random Forest
- XGBoost

הערכת הביצועים בוצעה באמצעות מספר מדדים מקובלים בתחום למידת המכונה ואבטחת המידע:

- Accuracy - אחוז הסיווגים הנכונים מתוך כלל הדוגמאות.
- Precision - אחוז התחזיות החיוביות שנמצאו נכונות.
- Recall - יכולת המודל לזהות מתקפות בפועל.
- F1-Score - מדד משולב של Precision ו-Recall.
- False Alarm Rate - שיעור אזעקות השווא, כלומר כמה פעמים המערכת זיהתה בטעות תעבורה תקינה כמתקפה.

לפני שלב הניקוי כלל מאגר הנתונים 918,448 רשומות. לאחר הסרת שורות עם ערכים לא חוקיים כגון NaN ו-Infinity, נותרו 917,117 רשומות תקינות לעבודה.

טבלת תוצאות המודלים:

מודל	Accuracy	Precision	Recall	F1-Score	False Alarm Rate
XGBoost	98.85%	98.87%	98.85%	98.86%	1.59%
Random Forest	98.83%	98.85%	98.83%	98.83%	1.66%
Logistic Regression	84.10%	86.06%	84.10%	83.42%	2.47%
Naive Bayes	80.87%	81.18%	80.87%	80.45%	9.41%

מהטבלה ניתן לראות כי אלגוריתם XGBoost השיג את הביצועים הטובים ביותר מבין כלל המודלים שנבדקו, עם דיוק של כ-98.85%. תוצאה זו קרובה מאוד לתוצאות שהוצגו במאמר המקורי, שבו דווח על דיוק של כ-99.11%.

בנוסף, ניתן לראות כי גם Random Forest השיג תוצאות גבוהות מאוד, אך XGBoost הצליח להגיע לדיוק מעט גבוה יותר ולשיעור אזעקות שווא נמוך יותר. לעומת זאת, המודלים Logistic Regression ו-Naive Bayes הציגו ביצועים נמוכים משמעותית, במיוחד במדדי F1-Score ו-Recall.

הבדלים קטנים בין תוצאות השחזור שלנו לבין תוצאות המאמר עשויים לנבוע מהבדלים בגרסאות ספריות, אופן ניקוי הנתונים, חלוקת הנתונים לאימון ובדיקה, או פרמטרים שלא פורטו באופן מלא במאמר המקורי.

בסיום ההשוואה נבחר XGBoost כמודל המוביל בעבודה, מאחר והוא הציג את השילוב הטוב ביותר בין דיוק גבוה לבין שיעור נמוך של אזעקות שווא.

ה. בעיות ואתגרים במהלך העבודה

במהלך ביצוע העבודה נתקלנו במספר אתגרים טכניים ומעשיים, אשר דרשו מאיתנו לבצע התאמות ושינויים על מנת להגיע לתוצאות הקרובות ככל האפשר למאמר המקורי.

הקושי הראשון היה הקמת סביבת העבודה. המאמר המקורי לא סיפק קוד מוכן או מימוש מלא של המערכת, ולכן נדרשנו לבנות את כל תהליך העבודה באופן עצמאי באמצעות שפת Python וספריות שונות כגון Scikit-Learn ו-XGBoost. בתחילת העבודה נדרש זמן רב להבנת מבנה המאגר, התאמת הספריות והגדרת סביבת עבודה יציבה.

אתגר משמעותי נוסף היה עבודה עם מאגר הנתונים CICIDS2017. המאגר כלל כמות גדולה מאוד של נתונים קרוב למיליון רשומות ולכן תהליך הקריאה, העיבוד והאימון של המודלים היה כבד יחסית מבחינת זמן ריצה ומשאבי מחשב. בנוסף, הנתונים כללו ערכים לא חוקיים כגון NaN ו-Infinity, אשר גרמו בתחילה לשגיאות במהלך האימון. כדי לפתור זאת, פיתחנו שלב ניקוי נתונים שביצע הסרה של רשומות בעייתיות ונרמול הנתונים באמצעות Z-score.

בעיה נוספת שבה נתקלנו הייתה חוסר אחידות בשמות העמודות במאגר. בחלק מהמקרים הופיעו רווחים מיותרים בתחילת שמות המאפיינים, דבר שגרם לשגיאות בזיהוי הפיצ'רים ובבחירתם מתוך הדאטה. לצורך פתרון הבעיה ביצענו ניקוי אוטומטי של שמות העמודות לפני תחילת העיבוד.

בנוסף, אחד האתגרים המרכזיים היה שחזור תוצאות המאמר בצורה נאמנה ככל האפשר. בתחילת העבודה התקבלו תוצאות שונות מהמאמר, ובפרט מודל Random Forest השיג ביצועים טובים יותר ממודל XGBoost, בניגוד למאמר המקורי. מצב זה דרש מאיתנו לבצע בדיקות חוזרות של הקוד, להבין את ההבדלים בין המודלים, ולבצע התאמות בפרמטרים ובמבנה המערכת.

במסגרת העבודה ביצענו מספר מטלות עיקריות:

- קריאה וניתוח של המאמר המקורי.
- הורדה והבנת מבנה מאגר הנתונים CICIDS2017.
- כתיבת קוד לעיבוד מקדים וניקוי הנתונים.
- בחירת המאפיינים המרכזיים בהתאם למאמר.
- מימוש ארבעת מודלי הלמידה השונים.
- חלוקת הנתונים לקבוצות אימון ובדיקה.
- חישוב מדדי ביצועים והשוואת תוצאות.
- ניתוח הפערים בין תוצאות השחזור לבין תוצאות המאמר.
- תיעוד העבודה והכנת הדוח והמצגת.

למרות הקשיים השונים, הצלחנו לבנות מערכת מלאה הפועלת בצורה דומה מאוד למערכת המתוארת במאמר, ולהגיע לתוצאות הקרובות לתוצאות המחקר המקורי.

ו. קוד ההרצה שלנו כולל הערות הסבר:

```

# Optional: Upload dataset files manually to Google Colab
#from google.colab import files
#uploaded = files.upload()

# Data manipulation libraries
import pandas as pd
import numpy as np

# Machine Learning utilities
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix

# Classification algorithms used in the experiment
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

# Load the two CICIDS2017 dataset files
file1 = "/content/Wednesday-workingHours.pcap_ISCX.csv"
file2 = "/content/Friday-WorkingHours-Afternoon-
DDos.pcap_ISCX.csv"

# Read CSV files into DataFrames
data1 = pd.read_csv(file1)
data2 = pd.read_csv(file2)

# Merge both datasets into a single DataFrame
data = pd.concat([data1, data2], ignore_index=True)

# Remove leading and trailing spaces from column names
data.columns = data.columns.str.strip()

# Display the number of rows before cleaning
print("Number of rows before cleaning: ", len(data))

# Convert the problem into binary classification:
# 0 = Benign traffic
# 1 = Attack traffic
data["Label"] = data["Label"].apply(

```

```

    lambda value: 0 if str(value).strip().upper() == "BENIGN" else 1
)

# Replace Infinity values with NaN
# Remove rows containing missing or invalid values
data.replace([np.inf, -np.inf], np.nan, inplace=True)
data.dropna(inplace=True)

# Display the number of rows after cleaning
print("Number of rows after cleaning: ", len(data))

# Remove irrelevant features according to the paper
columns_to_remove = [
    "Bwd PSH Flags",
    "Bwd URG Flags",
    "Fwd Avg Bytes/Bulk",
    "Fwd Avg Packets/Bulk",
    "Fwd Avg Bulk Rate",
    "Bwd Avg Bytes/Bulk",
    "Bwd Avg Packets/Bulk",
    "Bwd Avg Bulk Rate"
]

# Drop the selected columns if they exist
data.drop(columns=[col for col in columns_to_remove if col in
data.columns], inplace=True)

# Eight most important features selected in the paper
features = [
    "Fwd Packet Length Max",
    "Flow IAT Mean",
    "Fwd Packet Length Mean",
    "Flow IAT Std",
    "Fwd IAT Total",
    "Flow IAT Max",
    "Bwd Packet Length Std",
    "Bwd Packet Length Mean"
]

# Create feature matrix (X) and target vector (y)
X = data[features]
y = data["Label"]

```



```

# Split the dataset:
# 70% for training
# 30% for testing
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

# Apply Z-score normalization
scaler = StandardScaler()

# Fit the scaler on training data and transform both sets
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Calculate False Alarm Rate (FAR)
def calculate_far(y_true, y_pred):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    return fp / (fp + tn)

# Define the machine learning models to evaluate
models = {
    "Naive Bayes": GaussianNB(),

    "Logistic Regression": LogisticRegression(max_iter=1000),

    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        max_depth=10,
        random_state=42
    ),

    "XGBoost": XGBClassifier(
        n_estimators=200,
        max_depth=5,
        learning_rate=0.05,
        subsample=0.8,
        colsample_bytree=0.8,
        eval_metric="logloss",
        random_state=42
    )
}

```

```

    )
}

# Train and evaluate each model
results = []

for model_name, model in models.items():

    # Train the model
    model.fit(X_train, y_train)

    # Generate predictions on the test set
    predictions = model.predict(X_test)

    # Calculate evaluation metrics
    accuracy = accuracy_score(y_test, predictions)
    precision = precision_score(y_test, predictions,
                                average="weighted")
    recall = recall_score(y_test, predictions, average="weighted")
    f1 = f1_score(y_test, predictions, average="weighted")
    far = calculate_far(y_test, predictions)

    # Store the results
    results.append({
        "Model": model_name,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-Score": f1,
        "False Alarm Rate": far
    })

# Convert results into a DataFrame
results_table = pd.DataFrame(results)

# Sort models by accuracy (highest first)
results_table = results_table.sort_values(by="Accuracy",
                                           ascending=False)

# Display final comparison table
print("\nFinal Results: ")
print(results_table.to_string(index=False))

```

```
# Select the best-performing model
best_model = results_table.iloc[0]

# Display the best model name
print("\nBest Model: ")
print(best_model["Model"])

# Display the best accuracy score
print("\nBest Accuracy: ")
print(round(best_model["Accuracy"] * 100, 2), "%")
```

פלט המערכת :

```
Number of rows before cleaning: 918448
Number of rows after cleaning: 917117
```

Final Results:

Model	Accuracy	Precision	Recall	F1-Score	False Alarm Rate
XGBoost	0.988537	0.988667	0.988537	0.988550	0.015867
Random Forest	0.988326	0.988480	0.988326	0.988341	0.016618
Logistic Regression	0.841028	0.860636	0.841028	0.834163	0.024719
Naive Bayes	0.808713	0.811833	0.808713	0.804529	0.094137

```
Best Model:
XGBoost
```

```
Best Accuracy:
98.85 %
```

הקוד מממש מערכת לזיהוי מתקפות DoS/DDoS באמצעות למידת מכונה על מאגר הנתונים CICIDS2017. בתחילה הוא טוען שני קבצי נתונים, מאחד אותם ומבצע ניקוי נתונים הכולל הסרת ערכים חסרים או לא חוקיים. לאחר מכן כל סוגי המתקפות מאוחדים למחלקה אחת ("Attack"), בעוד שתעבורה רגילה מסומנת כ-"Benign", וכך מתקבלת בעיית סיווג בינארית.

בהמשך הקוד מסיר מאפיינים שאינם רלוונטיים, ובוחר שמונה מאפיינים מרכזיים שנקבעו במאמר כבעלי התרומה הגבוהה ביותר לזיהוי מתקפות. הנתונים מחולקים ל-70% אימון ו-30% בדיקה, ולאחר מכן מבוצע נרמול בשיטת Z-score כדי להביא את כל המאפיינים לאותה סקאלה.

המערכת מאמנת ארבעה מודלים שונים של למידת מכונה : Naive Bayes, Logistic Regression, Random Forest ו-XGBoost. עבור כל מודל מחושבים מדדי ביצועים כגון Accuracy, Precision, Recall, F1-Score ו-False Alarm Rate. לבסוף מוצגת טבלת השוואה בין כל המודלים, והקוד מדווח איזה מודל השיג את הביצועים הטובים ביותר על קבוצת הבדיקה. במקרה שלנו המודל המוביל היה XGBoost עם דיוק של כ-98.85%.

לפני הרצת הקוד יש להעלות לסביבת Google Colab (שאיתה עבדנו) את שני קובצי הנתונים מתוך מאגר CICIDS2017 שבהם נעשה שימוש בעבודה :

• Wednesday-workingHours.pcap_ISCX.csv

• Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv

הקוד מניח שקבצים אלו נמצאים בספריית העבודה של Colab (/content), ולכן יש להעלותם למערכת לפני תחילת ההרצה. לאחר העלאת הקבצים ניתן להריץ את הקוד ללא שינויים נוספים, והוא יבצע את כל שלבי עיבוד הנתונים, האימון וההערכה של המודלים בהתאם למתודולוגיה המתוארת במאמר.

ז. ביבליוגרפיה

המאמר המקורי (Primary Source):

Farhat, S., Abdelkader, M., Meddeb-Makhlouf, A., & Zarai, F.(2023) .

Evaluation of DoS/DDoS Attack Detection with ML Techniques on CIC-IDS2017 Dataset.

Proceedings of the 9th International Conference on Information Systems Security and Privacy (ICISSP 2023), pp. 287–295.

DOI: 10.5220/0011605700003405

קישור למאמר:

<https://www.scitepress.org/Papers/2023/116057/116057.pdf>

מאגר הנתונים (Dataset):

Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A.(2018) .

Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization.

Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP).

מאגר הנתונים CICIDS2017 פותח על ידי:

Canadian Institute for Cybersecurity (CIC), University of New Brunswick.

קישור למאגר:

<https://www.unb.ca/cic/datasets/ids-2017.html>

סביבת עבודה וקוד (Tools & Code):

הפרויקט מומש בשפת Python תוך שימוש בספריות:

- Pandas
- NumPy
- Scikit-Learn
- XGBoost

תיעוד XGBoost: https://xgboost.readthedocs.io/en/release_3.2.0